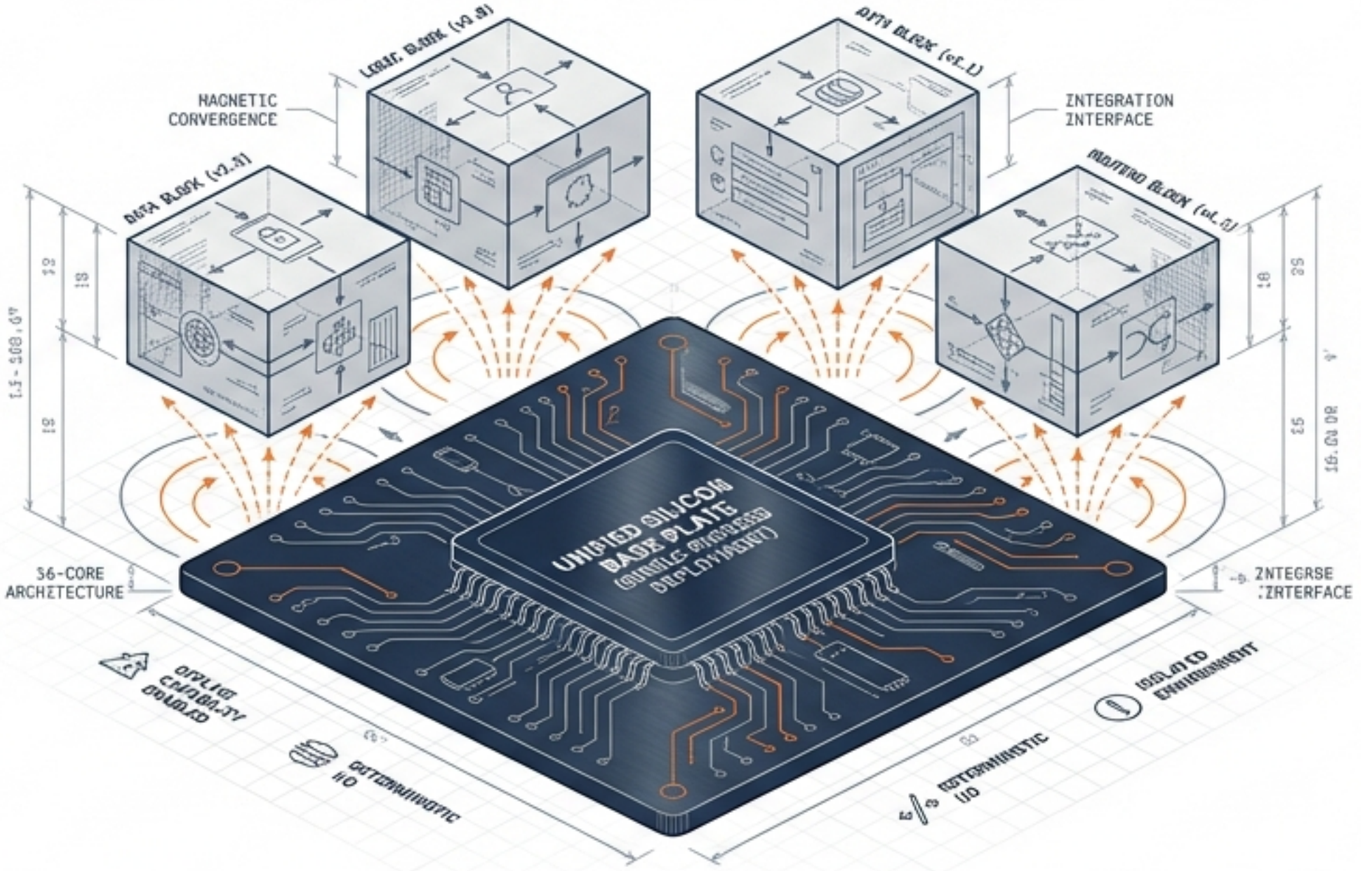
Standalone MITM Proxy Service

Architecting for Air-Gapped Environments & High-Velocity Testing



A deployment mode that collapses distributed microservices into a single process for offline capability and deterministic testing.

The Architectural Pivot: From Distributed to Embedded

The Standalone Mode allows the entire [redacted] **Proxy system**—normally a **cluster of microservices**—to run within a single process or container.

Integration Testing

Eliminate network flakiness.

Air-Gapped Operations

Isolated environments, no internet.

Simplified Prototyping

Low overhead demos.

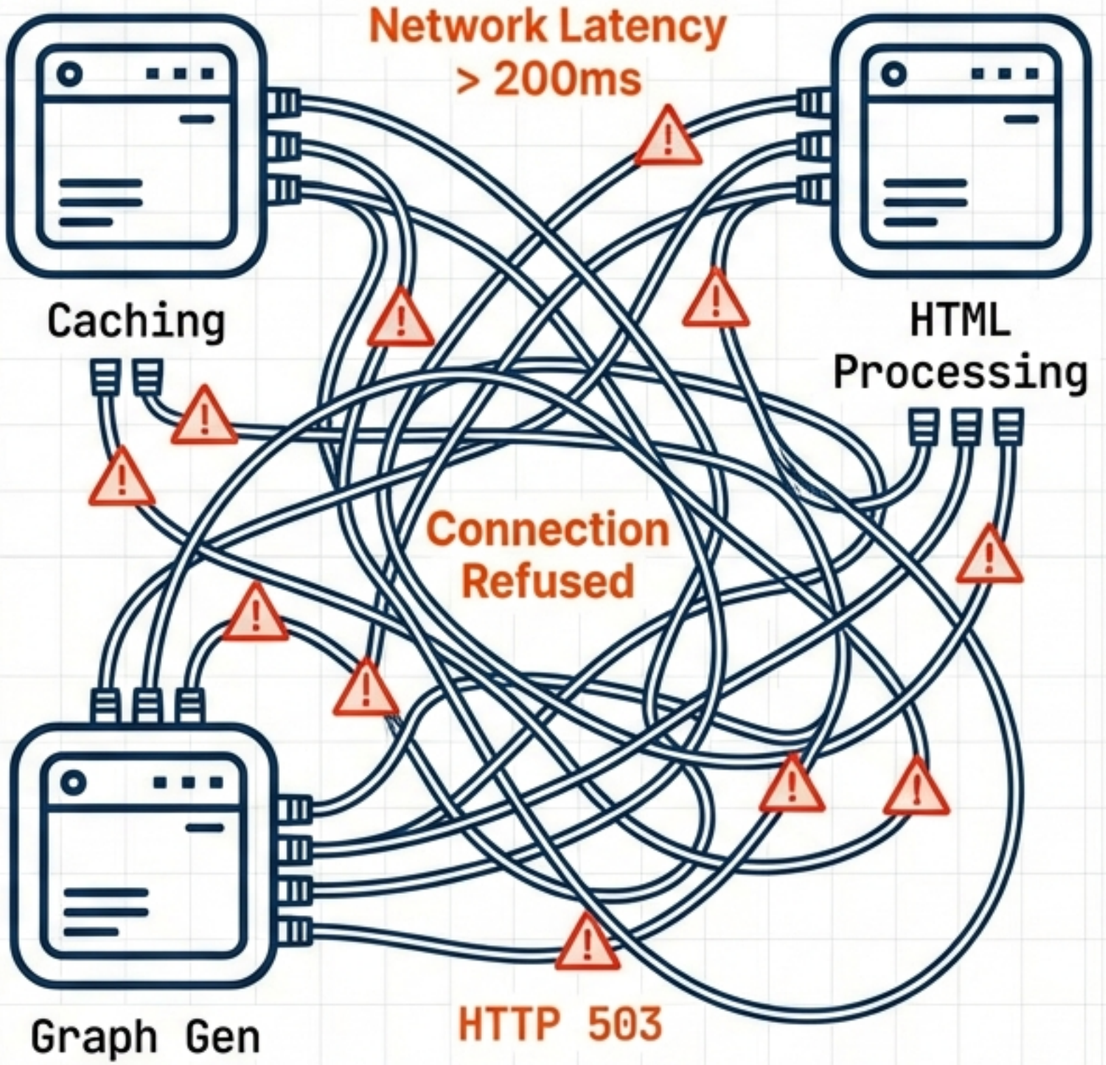
The Enabler:

Leveraging **FasTAPI** and **In-Memory Routing** to maintain service boundaries while **removing network latency**.

The Friction of Distributed Architectures

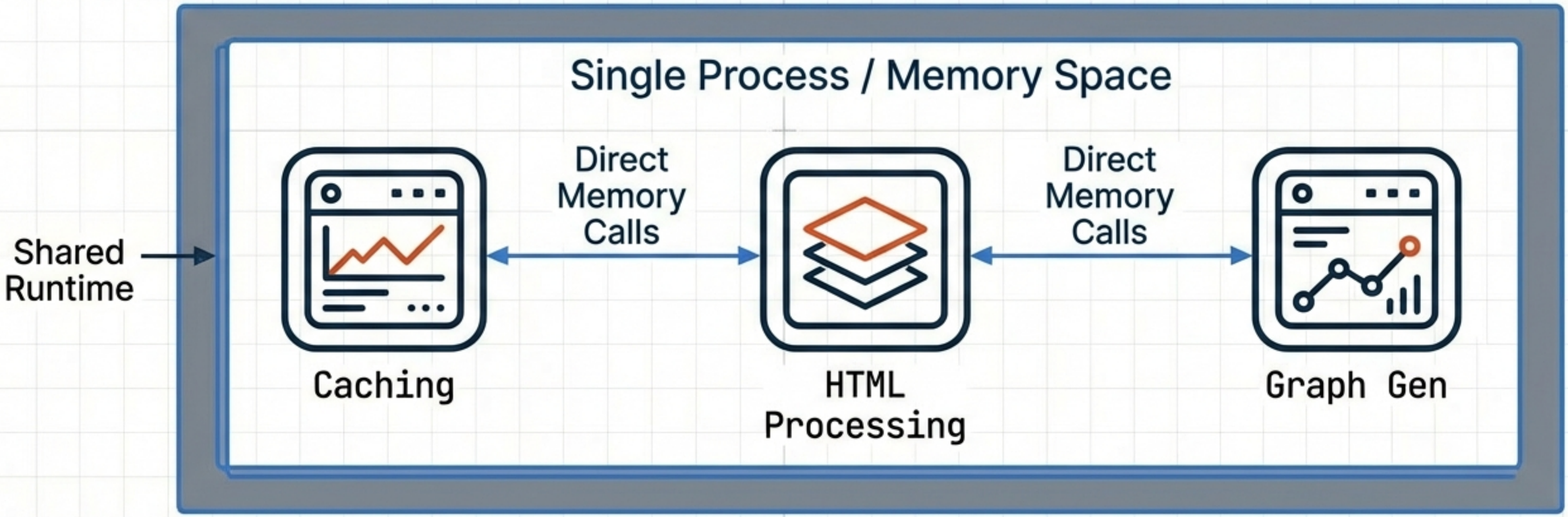
Context: Standard MITM Proxy platform.

- Integration Latency: Slow, non-deterministic tests due to network calls.
- Deployment Complexity: Managing multiple containers in air-gapped zones.
- Resource Overhead: High cost for simple prototypes.



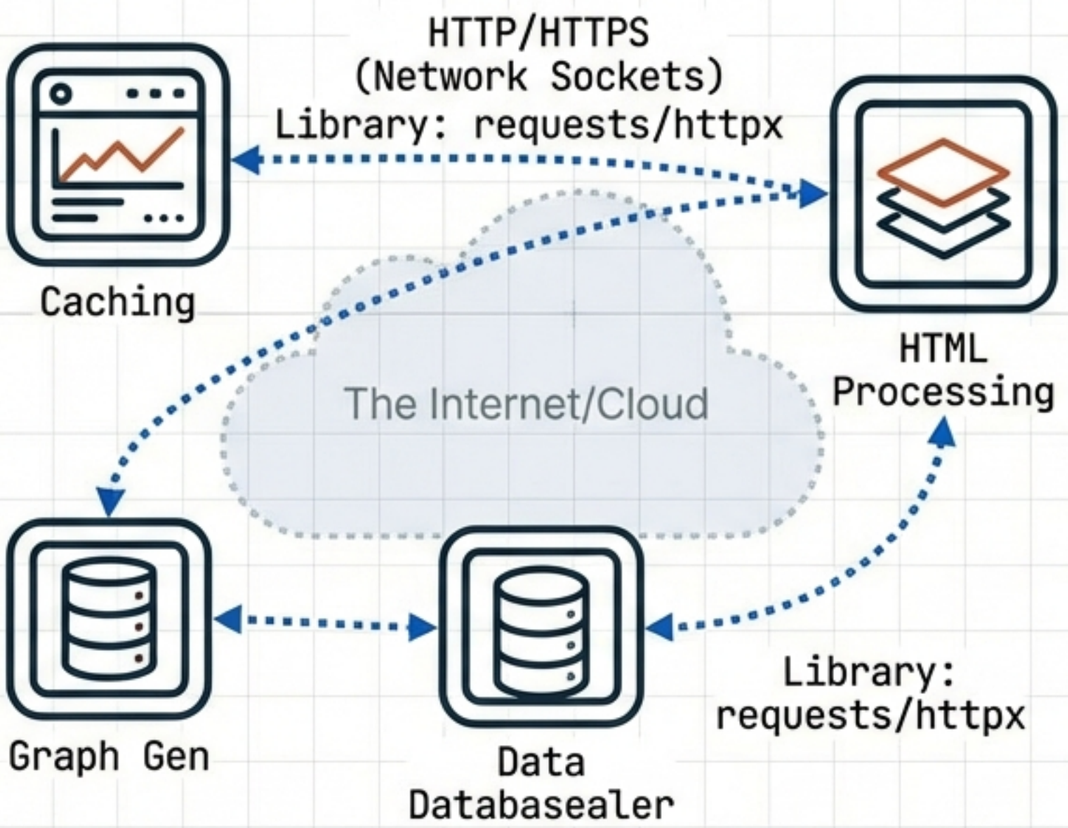
The Standalone Concept

A deployment configuration where all constituent microservices are instantiated as sub-applications within a single Python process. It is a miniature, fully functional version of the platform.

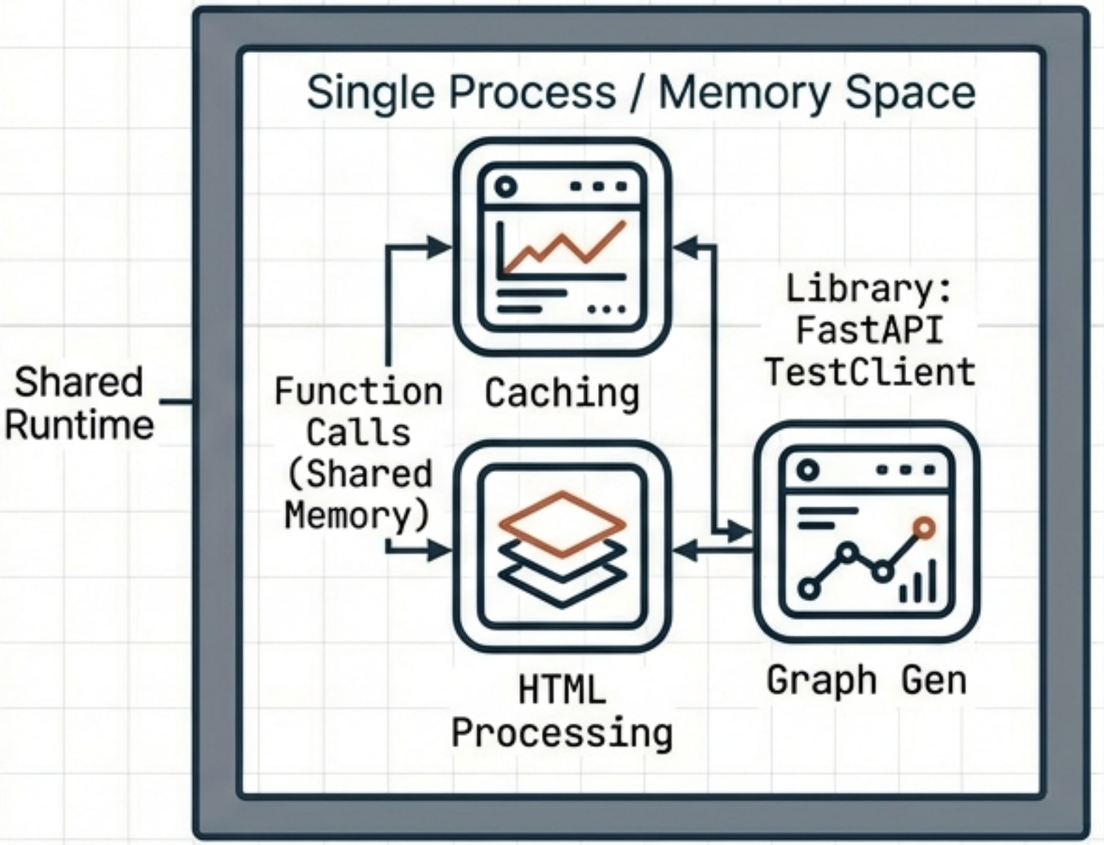


Architecture Visualized: Network vs. In-Memory

Production Mode (Distributed)

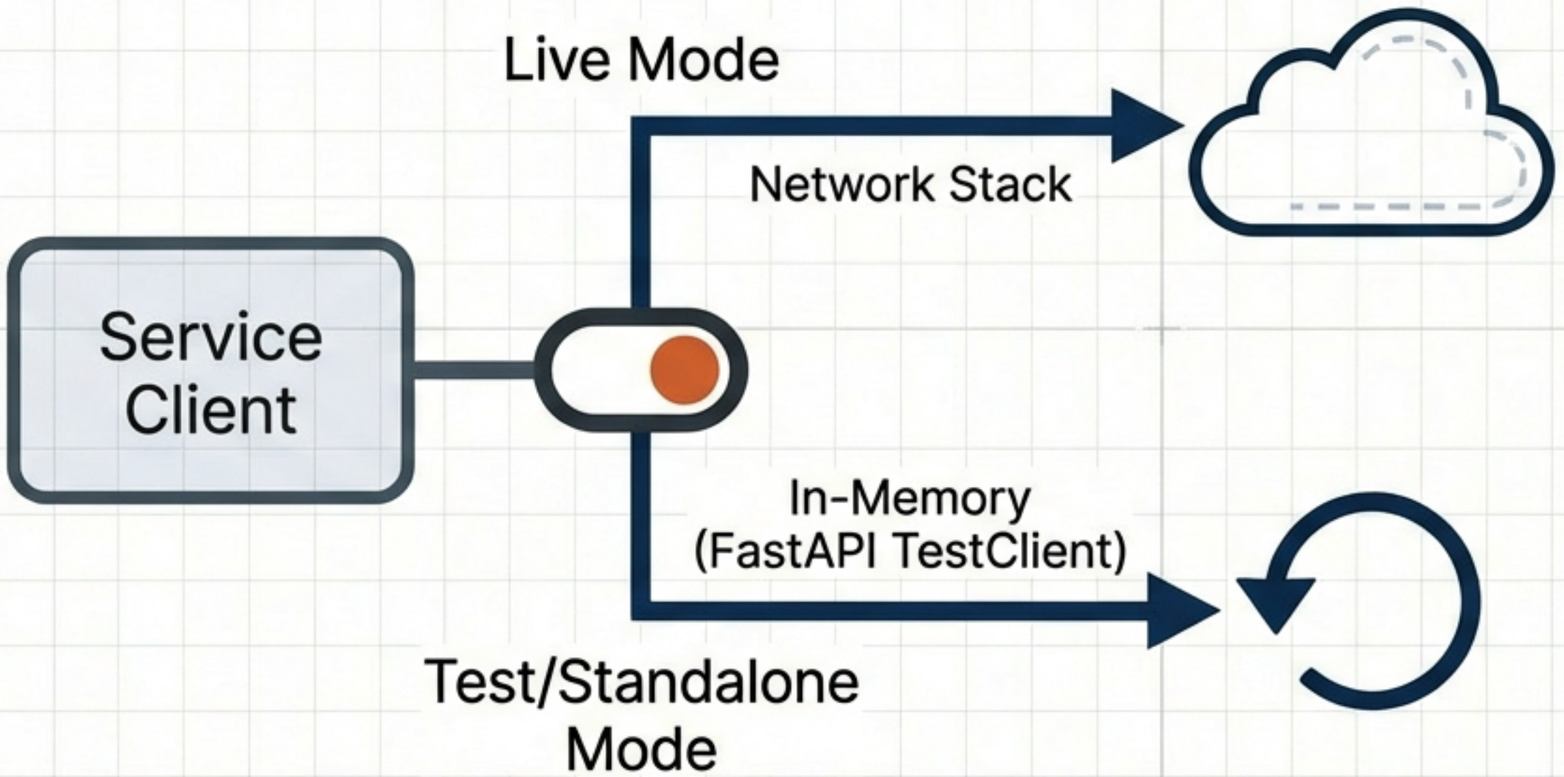


Standalone Mode (Embedded)



Logic remains identical. Transport layer changes.

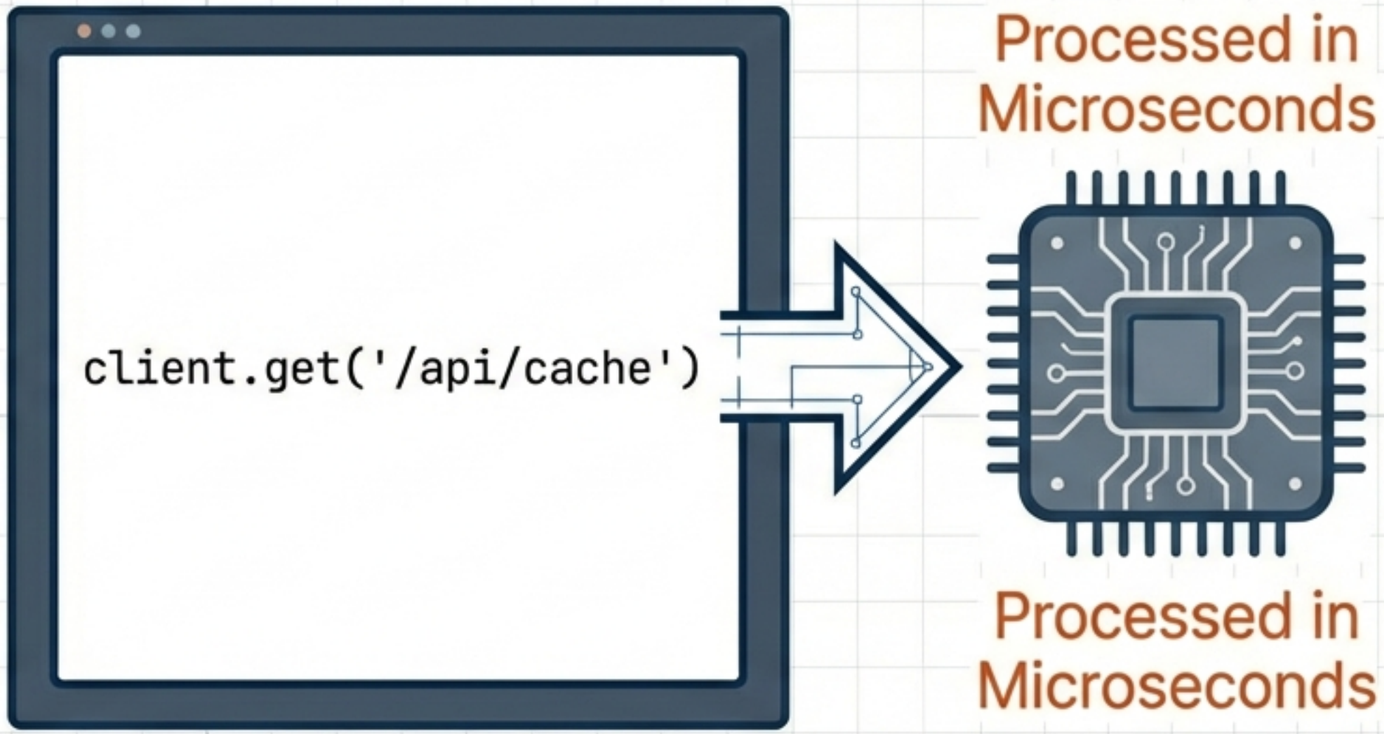
The Client Abstraction Layer



- Mode 1: Live HTTP Requests. Uses “requests” or “httpx” [redacted] Traffic traverses network.
- Mode 2: In-Memory Calls. Uses FastAPI TestClient [redacted] Bypasses network sockets entirely.

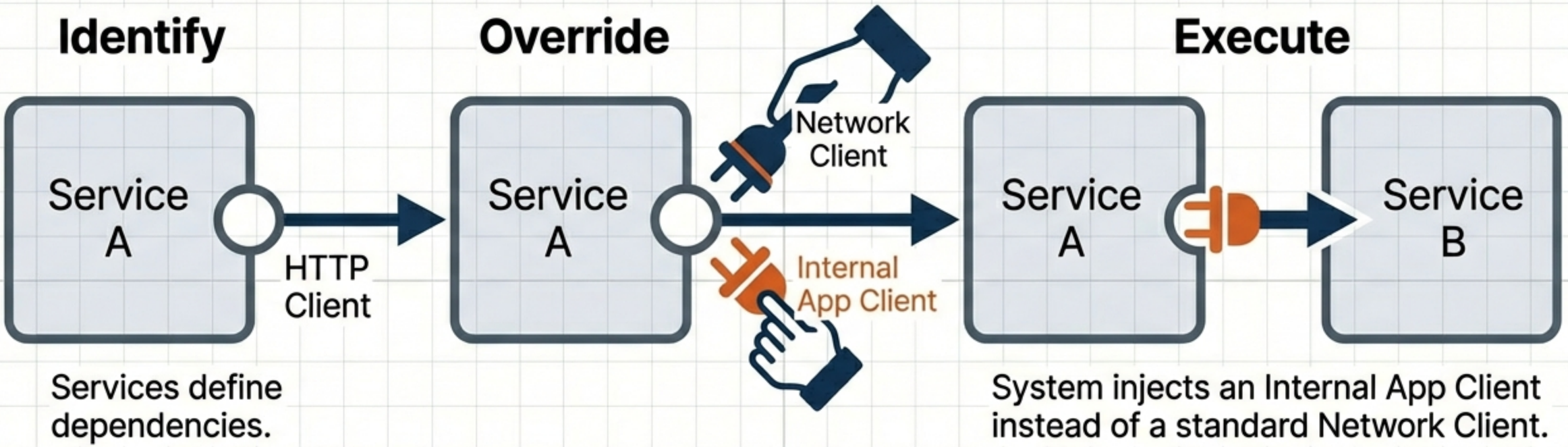
The Engine: FastAPI TestClient

- The `TestClient` allows the application to simulate HTTP interactions entirely in memory.
- It exposes the same interface as the `requests` library.
- Result: The service logic “thinks” it is making a network request, but the execution happens locally.



“The calls never leave the process.”

Dynamic Rewiring via Dependency Injection



- Services define dependencies. In standalone mode, the system injects an Internal App Client instead of a standard Network Client.
- Outcome: Zero code alteration in business logic.

The Startup Sequence

Total Initialization: < 1 Second

Initialize Services

Import and instantiate **FastAPI apps** as Python objects. Setup in-memory stores.

Configure Routing

Set clients to “Test Mode”. Establish internal object references.

Launch Proxy

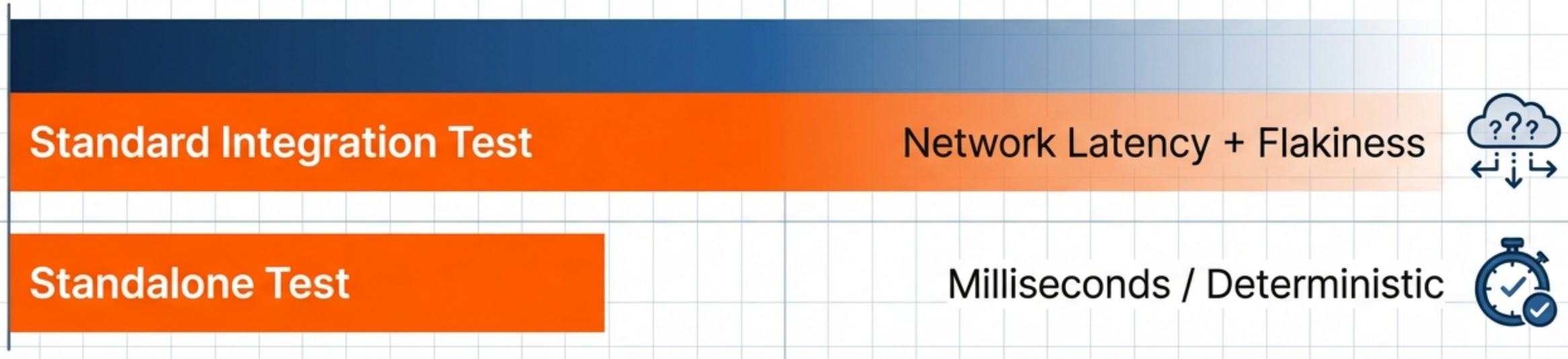
Start main application. No separate Uvicorn processes.

Total Initialization: < 1 Second

DOC-TECH-2024-003

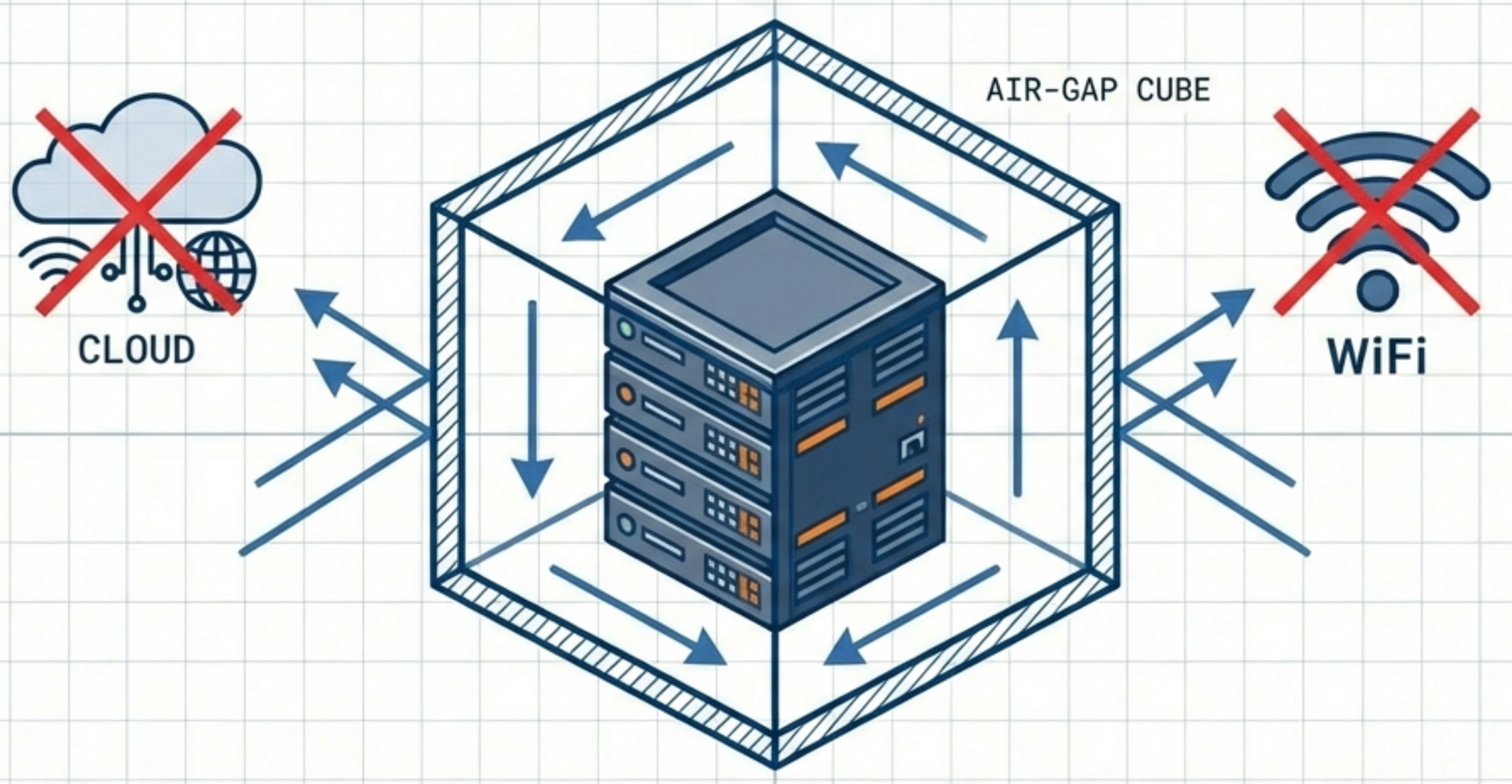
Use Case A: High-Velocity Integration Testing

Execution Time & Stability



- No mocking required: Exercises real code paths.
- Validates complex workflows (Proxy -> Caching -> HTML).
- Eliminates network IO flakiness.

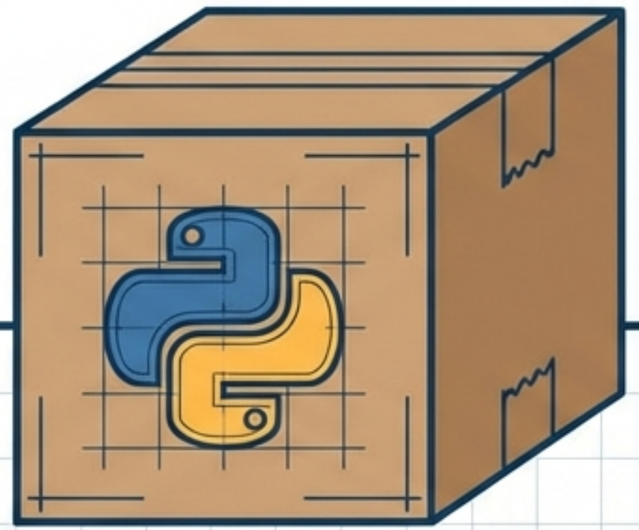
Use Case B: Air-Gapped & Offline Deployments



- Scenario: **High-security** environments or **edge devices**.
- Solution: **Single artifact**. Data never leaves the machine.
- Ideal for light usage where **Kubernetes** is overkill.

Packaging & Distribution

```
> pip install mitmproxy-standalone
```



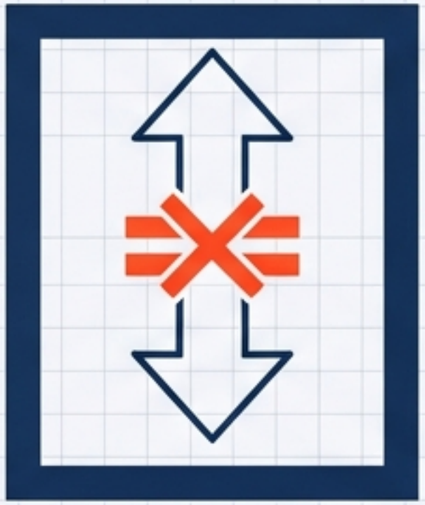
Format: Single Python package **(PyPI)** or **Docker Container**.

Dependencies: Python **runtime only**. No external DBs/Queues.

CI/CD: Built via standard **GitHub Actions** pipeline.

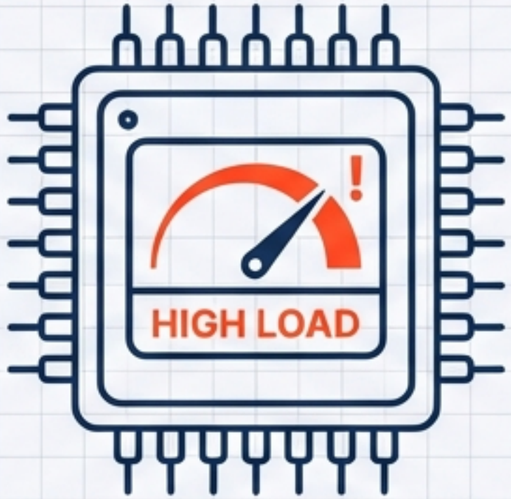
Architectural Limitations & Trade-offs

Scalability



Vertical scaling only. **Single process constraint.** No **horizontal scaling.**

Resources

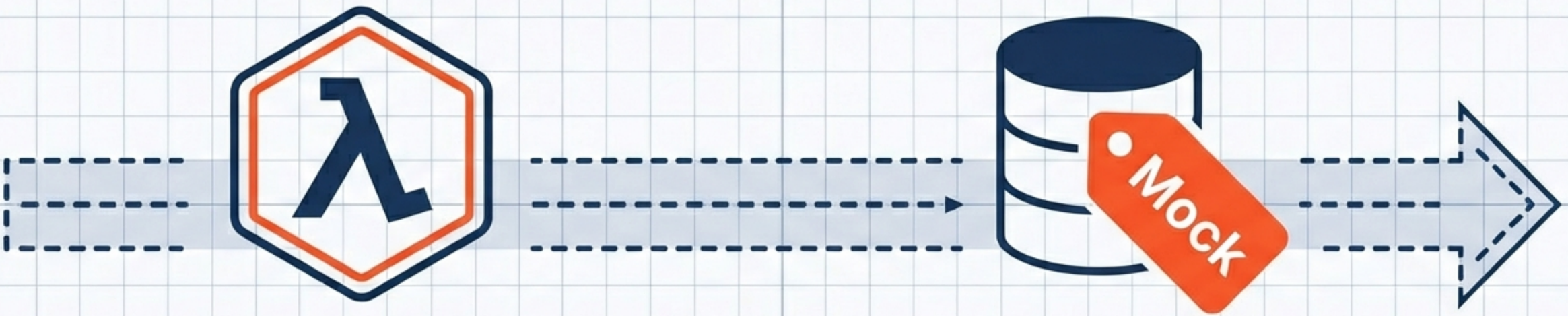


Shared CPU/Memory. **Heavy tasks block** all services.

Cloud Dependencies

External APIs must be reachable or mocked. Not fully offline if **3rd party APIs** are required.

Roadmap: Toward Total Encapsulation



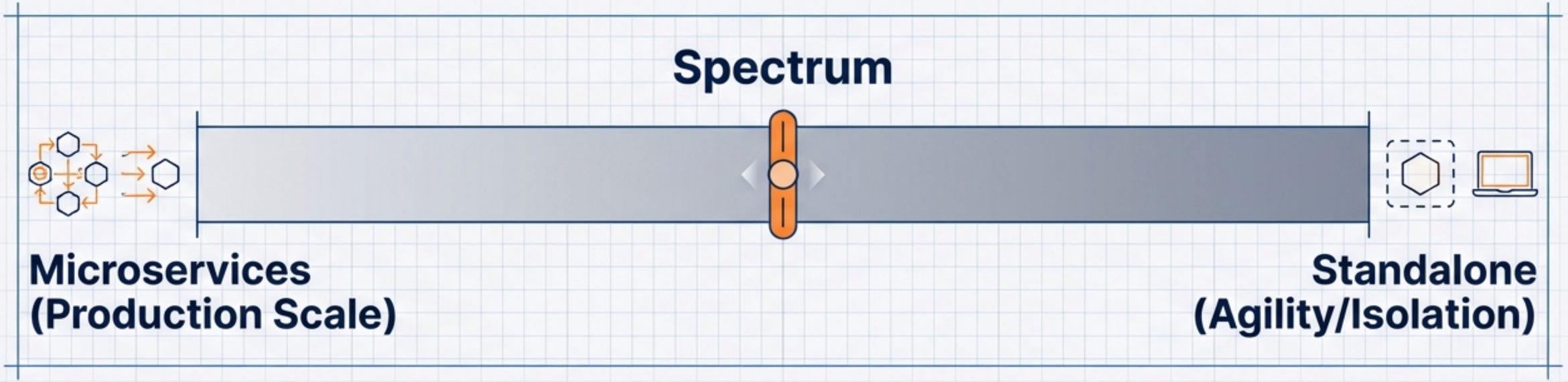
Serverless Potential.

Deploy unified app as **single function**.

Full Mocking.

Simulate all external cloud dependencies for **100% offline** capability.

Flexibility by Design



- **Summary:** Transforming testing friction into an architectural advantage.
- The same codebase adapts to the environment—running as independent clusters on **`Kubernetes`** or as a **`monolithic unit`** on an air-gapped laptop.